

Algorithms & Data Structures 1

Mid-term Coursework

Student Name: Bharath Raj Rajamohan
UOL Student ID: 230656932
SIM Student ID: 10256384

Table of Content

Section 1	Page 3
Section 2	Page 4
Section 3	Page 7
Section 4	Page 11
Section 5	Page 17
Section 6	Page 25

Section 1

The duty of a poetry assistant is to operate as a helping hand in assisting the poet by helping them suggest suitable and relevant rhymes for their poems. It should utilize a rhyme-suggestion algorithm to provide a list of words that rhyme with a given input word. For instance, if the poet needs suggestions on words that rhyme with the input word 'moon', the algorithm should output words such as "spoon", "soon", "tune", and "balloon". More advanced algorithms could be given the ability to understand that a poem follows a specific rhyme scheme such as 'ABAB', 'AAAA', or any other rhyme scheme. Program 2 that I have developed follows a similar logic however it is specialised in the 'ABAB' rhyme scheme. If the poet needs a hand at writing the final piece to the poem, the program will help to autofill a suitable word according to its reference word.

Section 2

Program 1

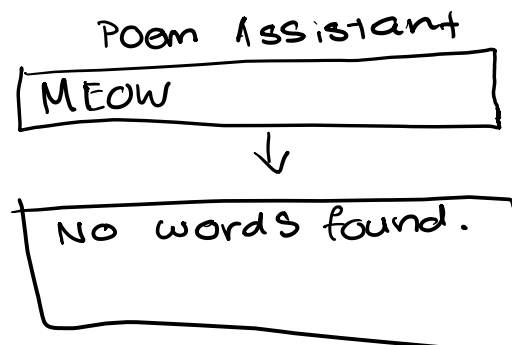
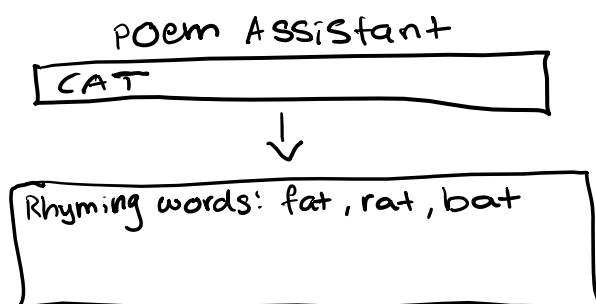
The first program's primary function is to find and display rhyming words for a given input word from a predefined collection. It serves as a testbed for me to create the simplest prototype possible of a poetic assistant. It operates by taking in an input word given by the user, and outputting a predefined list of words that are supposed to rhyme with the input word. If the user inputs an unrecognised word that is not part of the collection, it is also the duty of the program to respond by letting the user know that such a word is not present within the collection.

The first function of the program defines a collection of words predefined by me along with its set of rhyming words. For instance, the input word 'cat' is associated with words such as 'fat', 'rat', etc. as these are all rhyming words of cat. This can also be noted as a concise dictionary with only enough words needed to test the capabilities of program 1.

cat : fat , rat , Splat , bat , hat
dog : frog , cyborg , fog , log
moon : spoon , soon , tune , balloon

The second function of the program takes the users input and searches the predefined dictionary for rhyming words before displaying the results. It receives the word entered by the user in the input box and converts it to lowercase while removing any extra spaces. This is to ensure such mechanics of writing do not interfere with the execution of the program. Afterwards, the program takes the adapted input word and compares it with the words in the predefined collection to find a match. Once a match is found, the program takes the associated rhyming words and displays them into the output box. However, if no rhymes are shown, an appropriate message is shown.

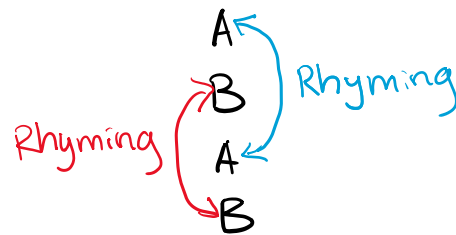
The final line within the program should show the event listener function for a button click. This line ensures that the function to find rhymes is called whenever the user clicks the "Find Rhymes" button. When the button is clicked, the function 'findRhymes' is triggered, and the program executes all the steps to process the input and display the results. The envisioned execution of Program 1 can be seen in the images below with the left image showing the output for a recognised input word and the right image showing the result of an unrecognisable input word.



Program 2

The second program allows the user to input a poem. The program processes the poem and replaces specific placeholders marked with '[?]' with words from a CSV file containing a dictionary significantly more comprehensive than that in program 1. These placeholders can be included in the poem to represent places where the user needs help finding a suitable word. The replacement words are then chosen based on the matching last 2 letters from the final word in the reference line in accordance with the 'ABAB' rhyme scheme.

Poem structure by lines:



The purpose of the first function is to load all the words from an external CSV file so that they are available for use within the program. It firstly, creates a request to read from the CSV file. If that's successful, the file contents are then processed and split into individual words. The individual words taken from the CSV file will all be stored into an array. Once loaded into an array, the program uses this array for all subsequent operations.

The purpose of the second function is to find a word from the array that ends with the same last two letters as the target word. The target word is decided based on the 'ABAB' rhyme scheme, meaning that each word must rhyme with the final word 2 lines down. When this function is executed, it traverses every word within the array and checks if the last two letters match the requirements. In order to identify what the last two letters of each word are, the slice method is used.

Example of slice in action:

- Suppose the word is 'night'.

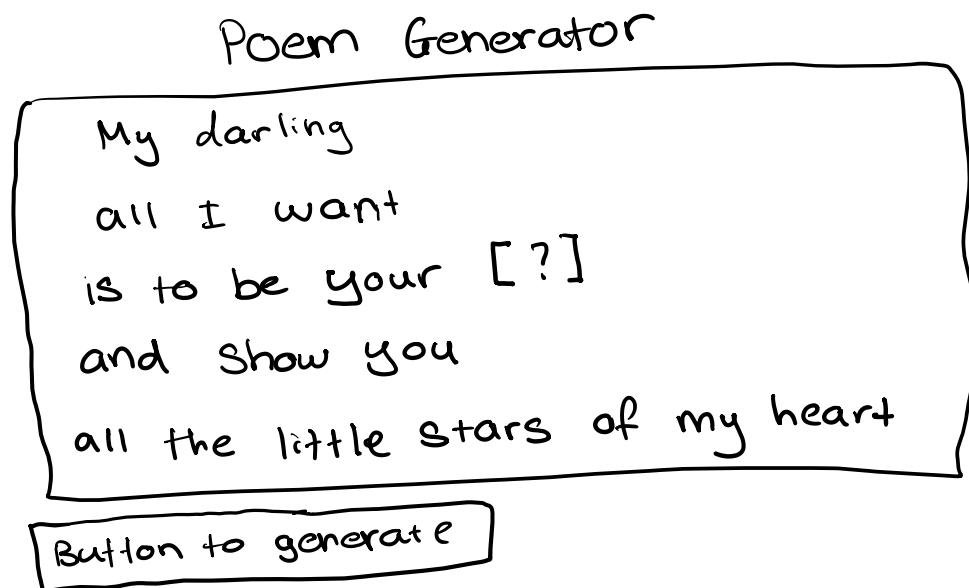
Using Slice:

- `Word.slice(-2)` will take the word "night" and extract the last two characters.
- In this case, it extracts "ht".

If a marching word is found, it is set as the replacement word and the search ends there. The function will then return the replacement word.

The third function then processes the poem, replacing the '[?]' placeholder used by the user to identify missing words with the replacement word found from the previous function. This is done by firstly, dividing the poem into individual lines using 'poem.split', the function then goes through the entire poem line by line while bookmarking any '[?]' placeholder found. Thereafter, it identifies the reference word for the rhyme and finds the replacement word matching the set parameters with the use of the previous function. The placeholder is then replaced with the new rhyming word and outputs the completed poem.

Similar to the one found in program 1, this final part of the program ensures that when the user clicks the "Find Rhymes" button, the poem is processed, and the placeholders are replaced. When the button is clicked, it captures the poem inputted by the user and calls upon the function to lead the catalog of words from the CSV file and add them into the array. The poem is then processed using the words from the array to replace any placeholder. The result is then displayed with the replaced words. An illustrated execution of program 2 can be seen below.



Output :

My darling
all I want
is to be your art → new word
and show you
all the little stars of my heart

Section 3

Pseudocode for Program 1

function findRhymes(inputWord)

 inputWord ← TOLOWERCASE(TRIM(inputWord))

if inputWord in rhymingWords **then**

 rhymes ← rhymingWords[inputWord]

 return "Rhyming words: " + JOIN(rhymes, ", ")

else

return "No rhymes found for this word."

end if

end function

function findRhymesButton.click()

 inputWord ← GETELEMENTBYID("wordInput").value

 outputBox ← GETELEMENTBYID("outputBox")

 outputBox.innerHTML ← findRhymes(inputWord)

end function

Pseudocode for Program 2

function loadWordsFromCSV(callback)

 xhr ← new XMLHttpRequest

OPEN(xhr, 'GET', '1000_most_used_words.csv', true)

function xhr.onload()

if xhr.status = 200 **then**

 wordsCSV ← xhr.responseText

 wordsArray ← **SPLIT**(wordsCSV, "\r?\n")

 callback(wordsArray)

else

PRINT("Failed to load CSV file.")

end if

end function

SEND(xhr)

end function

function findReplacementWord(targetWord, wordsArray)

 replacementWord ← null

for each word in wordsArray **do**

if **LENGTH**(word) ≥ 2 and **LENGTH**(targetWord) ≥ 2 and **SLICE**(word, -2) =
 SLICE(targetWord, -2) **then**

 replacementWord ← word

```
        break
    end if
end for
return replacementWord
end function

function processPoem(poem, wordsArray)
    lines ← SPLIT(poem, "\r?\n")
    replacedPoem ← " "

    for i ← 0 to LENGTH(lines) - 1 do
        line ← lines[i]

        if CONTAINS(line, "[?]") and i + 2 < LENGTH(lines) then
            nextLine ← lines[i + 2]
            words ← SPLIT(TRIM(nextLine), " ")
            lastWord ← REMOVE_PUNCTUATION(words[LENGTH(words) - 1])

            replacementWord ← findReplacementWord(lastWord, wordsArray)

            if replacementWord ≠ null then
                replacedPoem ← replacedPoem + REPLACE(line, "[?]",
                    replacementWord) + "\n"
            else
```

```
        replacedPoem ← replacedPoem + line + "\n"
    end if
else
    replacedPoem ← replacedPoem + line + "\n"
end if
end for

return TRIM(replacedPoem)
end function

function findRhymesButton.click()
    poem ← GETELEMENTBYID("poemInput").value
    outputBox ← GETELEMENTBYID("outputBox")

    if TRIM(poem) = "" then
        outputBox.innerHTML ← "Please enter a poem."
        return
    end if

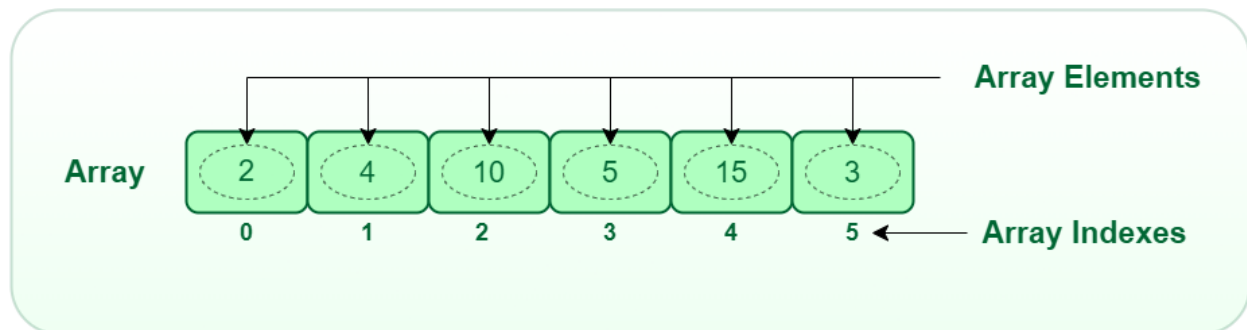
    loadWordsFromCSV(function(wordsArray)
        processedPoem ← processPoem(poem, wordsArray)
        outputBox.innerHTML ← "<br><strong>Processed Poem:</strong><br><pre>" +
            processedPoem + "</pre>"
    end function
```

Section 4

In developing the solutions to address the task of developing a poetry assistant, two primary data structures were pivotal: static array and a linear search algorithm. I have chosen to incorporate both as their inherent characteristics serve the best purpose in achieving my intended outcome.

Static Array

Figure 1:



Arrays were a necessity in enabling the functionality of program 1. They are effective for storing ordered lists of elements, making them ideal for holding collections of rhyming words associated with each word. Since my intention while developing program 1 was for it to be the most basic form of a poetry assistant to utilise it as a sturdy foundation while developing upon it for program 2, I had decided to manually input certain words and a handful of words rhyming to them. This way, I had the most controlled testbed where there was no uncertainty in the accuracy of the suggested rhyming words.

As such, I had concluded to tap into the utility that an array offers. Within the array, I had stored a handful of rhyming words associated with input words such as "cat", "dog", and "moon". Figure 1 gives us a visualization of how an array operates. Each element within the array is assigned an index which the program can call upon to retrieve the elements. Having an array gave me the option on how I wanted to design the retrieval of all rhyming words associated with an input word. I had the option of either calling upon certain indexes to retrieve a specific word or I could retrieve all the elements at once as I have done.

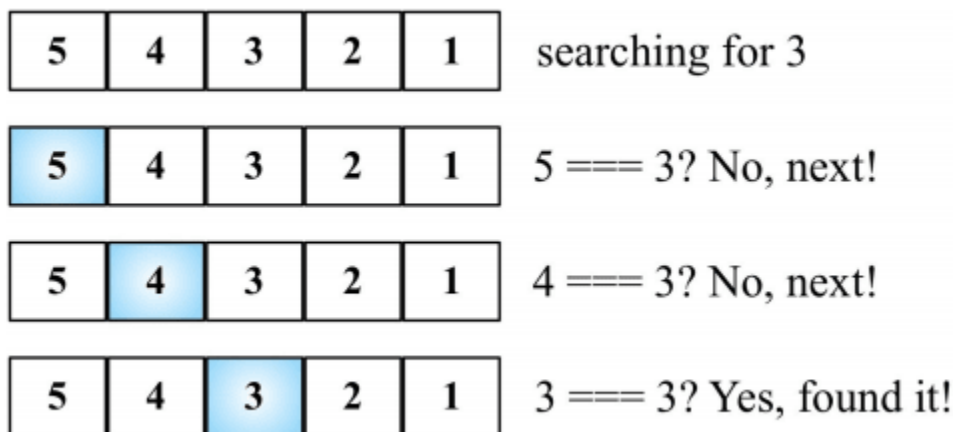
However, arrays inherently also have certain limitations built into them. Firstly, they have a fixed size where their capacity must be defined at the time of creation. This can lead to insufficient capacity if the number of elements changes dynamically. Another limitation is that arrays are not efficient in handling operations like searching for an element or removing specific elements in large arrays, especially if the array is not sorted.

Regardless, I had still chosen to incorporate a static array as the pros clearly outweighed the cons. As for program 1, the rhyming words for specific inputs such as cat and moon are predefined, and will not be expanded to preserve its simplicity as its purpose is to be a simplest form of a poetry assistant. Whereas for program 2, the number of elements loaded from the CSV

file will always be fixed to 1000 as the file only contains 1000 words. Therefore, the resizable nature of dynamic arrays is not needed as the words are loaded into the array once and remain unchanged. Moreover, static arrays are beneficial for both programs as they allow for direct access to elements via indexing. This allows for efficient retrieval of words from the array. Lastly, the fixed array size of static arrays allows for more straightforward and predictable memory allocation which results in faster execution of the program.

Linear Search Algorithm

Figure 2:



In Program 2, a linear search algorithm plays a crucial role in efficiently finding replacement words within a loaded list of words. It is a type of sequential searching algorithm in which every element within the array is traversed and compared to find a word matching my preset parameters.

Program 2 is made with the intention of crafting an algorithm where the final missing word to a poem can be found. It is designed to specialise in auto filling poems that follow the ABAB rhyme scheme. Due to its increased complexity as opposed to program 1, it has access to a larger list of words.

A linear search algorithm was chosen as the program needs to follow a predetermined parameter that the replacement word must share the same last 2 letters with the reference word within the ABAB rhyme scheme. As such, the algorithm sieves through the loaded list of words looking for a word that satisfies this criterion.

The linear search algorithm also offers scalability and flexibility in the sense that it does not require the loaded dataset to be sorted or indexed, allowing me to upsize or edit the data set as I see fit without the worry of the program failing consequently.

However, this data structure has its own limitations. Linear search has $O(n)$ time complexity, making it slow for large datasets as it may need to check every element. This also limits its scalability as there is a limit to the size of the dictionary that is loaded. Regardless of these

limitations, I chose to implement linear search as I had planned for my program to only have access to 1000 words which based on testing, did not cripple the algorithms capabilities.

Vectors

Vectors are a type of data structure that provides a flexible and efficient way to manage a collection of elements. Unlike static arrays, vectors can dynamically resize themselves to accommodate the addition and removal of elements. Vectors use a contiguous block of memory to store elements, similar to arrays. However, when the capacity of the vector is reached, it allocates a larger block of memory, copies the existing elements to the new block, and continues to add new elements. This process of resizing ensures that vectors can handle varying amounts of data efficiently.

In terms of the advantages of using vectors, as aforementioned they can automatically resize themselves when they reach capacity, eliminating the need to manually allocate and deallocate memory. They are also suitable for scenarios requiring fast random access to elements thanks to their constant-time access ($O(1)$) to elements using an index. Moreover, they are compatible with a wide range of operations such as sorting, searching, and transforming elements.

The cons of using them are that when a vector resizes, it must allocate a new block of memory and copy the existing elements to the new block. This process can be time-consuming when performed frequently. Additionally, Inserting or deleting elements at positions other than the end of the vector requires shifting elements to maintain contiguity, causing $O(n)$ time complexity for these operations which can be inefficient for larger vectors.

In the case of program 1, I deemed that vectors would be unsuitable as program 1's primary task is to find and display rhyming words for a given input word from a predefined and static collection. There is no need for dynamic resizing as program 1's purpose is to merely only act as a prototype with limited functionality. Therefore, the collection does not change during the execution of the program. Moreover, the program does not involve frequent insertion or deletion of elements. Since the collection of rhyming words remains unchanged, the overhead associated with the dynamic resizing and shifting of elements in vectors is not justifiable.

In regards to program 2, I chosen not to introduce vectors instead of linear search as the list of words found in the dictionary are read once and stored in an array for processing. They do not change during the execution, so dynamic resizing is not required. Moreover, a search operation is needed to identify a word that satisfies the search parameters. Thus, a linear search is more suited to my programs needs than vectors.

Stacks

Stacks are a type of data structure that follows the Last In, First Out (LIFO) principle where the last element added to the stack is the first one to be removed. It supports push and pop operation. A push operation adds an element to the top of the stack while pop removes an element from the top of the deck.

The advantages associated with stacks are that they are well-suited for managing recursive function calls. Each call is pushed onto the stack, and when a function returns, it is popped from the stack. This recursion simplifies the implementation of algorithms like depth-first search. Additionally, stacks are ideal for backtracking algorithms, where you need to explore all possible options and then return to a previous state. Lastly, in applications like text editors, stacks provide an efficient way to implement undo functionality. Each action is pushed onto the stack, and an undo operation pops the last action to revert it.

Stacks also come with multiple disadvantages. Unlike arrays or vectors, stacks do not support direct indexing of elements. Hence, the elements must be popped sequentially until the desired element is reached. Additionally, stacks have single-use storage. Elements are typically only used once in stacks as once they are popped, they are removed from the stack. This characteristic makes stacks unsuitable for creating a poetry assistant as the program is not intended to be a single-use program.

In the case of program 1, stacks were not chosen as it does not support random access, making it inefficient to retrieve and display all rhyming words. Arrays, on the other hand, provide constant-time access to elements, making them more suitable. Moreover, the collection of rhyming words is static. There is no need for the LIFO behavior provided by stacks, as elements are not added or removed during the program's execution. Lastly, the program needs to retrieve and display all rhyming words at once. Stacks are designed for accessing the most recent elements and not for bulk retrieval of all elements.

In regards to program 2, the words from the dictionary file are read once and stored in an array for processing. The list of words does not change during the execution, so the dynamic nature of stacks is unnecessary. Similarly to the rationale mentioned for vectors, program 2 involves searching for a rhyming word based on the preset parameters. A linear search algorithm will thus be a more fitting data structure to serve this objective. Lastly, stacks only allow access to the top element, making it inefficient for operations that require accessing and modifying multiple elements. Program 2 needs to process the entire poem and search through all words, which is not feasible with the LIFO constraint of stacks. Hence, stacks were not deemed as a beneficial addition to program 2.

Queues

Unlike stacks, queues follow the First In, First Out (FIFO) principle. This means that the first element added to the queue is the first one to be removed. It is a collection of elements that

supports two operations; Enqueue which adds an element to the end of the queue and Dequeue which removes an element from the front of the queue.

Some of the advantages of queues are that they preserve the order in which the elements were added. Additionally, it's FIFO nature ensures that every element gets the chance to be processed, thus preventing starvation. Queues are also useful in multi-threaded applications where tasks need to be processed in the order they arrive.

However, queues also harbor some disadvantages. Firstly, there is limited access due to the FIFO principle as accessing elements other than the front requires dequeuing all elements before it. Its absence of indexing further supports this point as it is not possible to access a specific element through its index without dequeuing all other elements sequentially. Additionally, queues have a single use storage like stacks. Once certain elements had been dequeued to access a specific element, the dequeued elements can no longer be retrieved. This characteristic is unfavorable to a poetry assistant program that needs to access its dictionary over multiple instances.

Queues were unsuitable for program 1 as the program requires direct access to the list of rhyming words for a given input word. Queues do not support random access, making it inefficient to retrieve and display all rhyming words. Moreover, the program needs to retrieve all rhyming words at once. Queues are designed for processing elements in the order they are added, not for bulk retrieval of all elements. Lastly, since the set of chosen rhyming words are predefined and unchanging, queue's FIFO nature is unnecessary as elements are neither added nor removed. For these reasons, incorporating queues would not have been beneficial for program 1.

As for program 2, the words from the CSV file are read only once and stored within the array. Thus, the dynamic nature of arrays is unnecessary as the list of available words will not change during the program's execution. Moreover, the core of program 2's execution is the ability to search for words following certain parameters. This is best handled by linear search algorithm instead of queues. Lastly, program 2 also requires replacing placeholders within the poem, which requires direct access to specific elements which queues are not efficient in and would dequeue a large portion of the words found in the CSV file after only once execution which is unfavourable.

Dynamic arrays

Dynamic arrays are a type of data structure that can grow and shrink in size automatically, allowing for flexible memory management. Unlike static arrays, which have a fixed size determined at the time of creation, dynamic arrays can adjust their size as needed when elements are added or removed.

As aforementioned, one of the benefits of dynamic arrays is that they can grow and shrink as needed which is beneficial for scenarios where the numbers of elements are uncertain and can change frequently. Moreover, they allow access to elements by index which allows for quick access to elements. Lastly, since dynamic arrays allocate memory as needed, the chances of memory wastages are minimized.

As for its disadvantages, dynamic arrays incur time cost when resizing to allocate for the new array and copy the elements from the old array to the new. Additionally, it is space inefficient. To reduce the frequency of resizing, dynamic arrays often allocate more memory than is currently needed. This can lead to some wasted space.

My rationale for now utilising dynamic arrays for program 1 is that firstly, the set of rhyming words does not change during the program's execution. Hence, there is no need for the dynamic resizing capabilities of dynamic arrays. Additionally, as mentioned earlier, dynamic arrays have the tendency to allocate more memory than necessary to handle potential growth in the array size. This extra memory usage may consume more system resources than necessary and reduce the efficiency of the program. Lastly, static arrays provide the required random access to elements. Therefore, the additional overhead associated with dynamic arrays are not justifiable.

Linked list

Linked lists are a form of data structure where elements, called nodes, are linked together using pointers. Each node contains data and a reference to the next node in the sequence.

The strengths of linked lists are that they can grow and shrink in size dynamically which is beneficial when the number of elements is not known in advance. Secondly, insertions and deletions of nodes can be performed efficiently when adding or removing elements at the beginning or end of the list. Lastly, linked lists allocate memory only for the elements currently in the list. Therefore, using only as much memory as needed unlike dynamic arrays.

As for the drawbacks, they do not allow access to elements using index. Instead, to access an element, it is required to traverse the list from the beginning which causes the program to take longer to execute than would otherwise be needed. Additionally, each node in a linked list requires additional memory for storing the reference to the next node. In doubly linked lists, even more memory is needed to store references to both the next and previous nodes.

Compared to linked lists, arrays would be a simpler and more straightforward implementation for managing a static collection of words found in program 1. As for program 2, linked lists happen to also fall short of linear search algorithm as utilizing them would result in slower search times due to sequential access, which is unfavourable. Moreover, program 2 requires replacing placeholders in the poem with matching words. This process benefits from the more efficient access provided by arrays, whereas linked lists would overcomplicate the implementation without offering any performance improvements.

Section 5

Program 1 (JavaScript)

```
// Object containing all predetermined rhyming words for given inputs
const rhymingWords = {
  "cat": ["fat", "rat", "splat", "bat", "hat", "chat"],
  "dog": ["frog", "cyborg", "fog", "log", "jog"],
  "moon": ["spoon", "soon", "tune", "balloon"],
};

// Function to find a rhyme
function findRhymes() {
  // Get the input word from the input field, convert it to lower case, and
  trim any extra spaces
  const inputWord =
document.getElementById("wordInput").value.toLowerCase().trim();
  // Get the output box element
  const outputBox = document.getElementById("outputBox");

  // Check if the input word exists in the rhymingWords object
  if (rhymingWords[inputWord]) {
    const rhymes = rhymingWords[inputWord];
    // Display the rhyming words in the output box
    outputBox.innerHTML = `Rhyming words:</strong>
${rhymes.join(", ")}`;
  } else {
    //output message if no rhyming words are found
    outputBox.innerHTML = "<strong>No rhymes found for this
word.</strong>";
  }
}

// button to call findRhymes function when clicked
document.getElementById("findRhymesButton").addEventListener("click",
findRhymes);
```

Program 1 (HTML 5)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Rhyme Finder</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      padding: 20px;
    }
    .container {
      max-width: 50;
      margin: 0;
    }
  </style>
</head>
<body>
  <div class="container">
    <div class="input">
      <input type="text" value="wordInput" />
    </div>
    <div class="button">
      <button value="findRhymesButton" />
    </div>
    <div class="output">
      <div value="outputBox" />
    </div>
  </div>
</body>
</html>
```

```

    }
    label {
        font-weight: bold;
    }
    input[type="text"], textarea {
        width: 100%;
        padding: 8px;
        margin-top: 5px;
        margin-bottom: 10px;
        border: 1px solid #ccc;
        border-radius: 4px;
        box-sizing: border-box;
    }
    button {
        padding: 10px 20px;
        background-color: #4CAF50;
        color: white;
        border: none;
        border-radius: 4px;
        cursor: pointer;
    }
    button:hover {
        background-color: #45a049;
    }
    .output {
        margin-top: 10px;
        padding: 10px;
        border: 1px solid #ccc;
        border-radius: 4px;
        min-height: 100px;
    }
}
</style>
</head>
<body>
    <small>Name: Bharath Raj Rajamohan</small>
    <br>
    <small>UOL Student ID: 230656932 </small>
    <br>
    <small>SIM Student ID: 10256384</small>
    <div class="container">
        <h2>Rhyme Finder</h2>
        <label for="wordInput">Enter a word:</label>
        <input type="text" id="wordInput" placeholder="Enter your word...">
        <button id="findRhymesButton">Search</button>
        <div class="output" id="outputBox"></div>
    </div>

    <!-- Link to external JavaScript file -->
    <script src="program-1.js"></script>
</body>
</html>

```

Program 2 (JavaScript)

```
// Function to load CSV file and extract words
function loadWordsFromCSV(callback) {
    const xhr = new XMLHttpRequest();
    xhr.open('GET', '1000_most_used_words.csv', true);

    // When the request is loaded
    xhr.onload = function() {
        if (xhr.status === 200) {
            const wordsCSV = xhr.responseText;
            // Split by lines and extract words
            const wordsArray = wordsCSV.split(/\r?\n/);
            callback(wordsArray); // Call the callback function with the
words array
        } else {
            console.error('Failed to load CSV file.');
```

```
        }
    };
}
```

```
xhr.send(); // Send the request
```

```
}
```

```
// Function to find a replacement word based on last two letters
```

```
function findReplacementWord(targetWord, wordsArray) {
```

```
    let replacementWord = null;
```

```
    // Iterate over each word in wordsArray
```

```
    wordsArray.some(word => {
```

```
        // Check if the last two letters of the words match
```

```
        if (word.length >= 2 && targetWord.length >= 2 &&
```

```
            word.slice(-2).toLowerCase() === targetWord.slice(-
2).toLowerCase()) {
```

```
            replacementWord = word;
```

```
            return true; // Exit loop once a matching word is found
```

```
        }
```

```
        return false; // Continue loop if no match
```

```
    });
```

```
    return replacementWord; // Return the found replacement word, or null if
not found
```

```
}
```

```
// Function to process the poem and find a replacement for [?]
```

```
function processPoem(poem, wordsArray) {
```

```
    // Split the poem into lines
```

```
    const lines = poem.split(/\r?\n/);
```

```
    let replacedPoem = '';
```

```
    // Iterate over each line of the poem
```

```
    for (let i = 0; i < lines.length; i++) {
```

```
        const line = lines[i];
```

```
        // Check if the line contains '[?]' and there are at least two lines
ahead
```

```
        if (line.includes('[?]' && i + 2 < lines.length) {
```

```
            const nextLine = lines[i + 2];
```

```
            const words = nextLine.trim().split(" ");
```

```

        const lastWord = words[words.length - 1].replace(/([^\w\s]|_)/g,
        ""); // Remove punctuation

        // Find a replacement word from wordsArray where last two letters
match
        const replacementWord = findReplacementWord(lastWord,
wordsArray);

        // If a replacement word is found, replace '[?]' with it
        if (replacementWord) {
            replacedPoem += line.replace('[?]', replacementWord) + '\n';
        } else {
            replacedPoem += line + '\n'; // If no replacement found, keep
the original line
        }
    } else {
        replacedPoem += line + '\n'; // If line doesn't contain '[?]',
keep it as is
    }
}

return replacedPoem.trim(); // Return the processed poem
}

// Function to handle button click event
document.getElementById("findRhymesButton").addEventListener("click",
function() {
    const poem = document.getElementById("poemInput").value;
    const outputBox = document.getElementById("outputBox");

    // Check if the poem input is empty
    if (poem.trim() === "") {
        outputBox.innerHTML = "<strong>Please enter a poem.</strong>";
        return;
    }

    // Load words from CSV and process the poem
    loadWordsFromCSV(function(wordsArray) {
        const processedPoem = processPoem(poem, wordsArray);
        // Display the processed poem in the output box
        outputBox.innerHTML = `<br><strong>Processed
Poem:</strong><br><pre>${processedPoem}</pre>`;
    });
});

```

Program 2 (HTML 5)

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Poetry Assistant</title>
</head>

```

```
<body>
  <small>Name: Bharath Raj Rajamohan</small>
  <br>
  <small>UOL Student ID: 230656932 </small>
  <br>
  <small>SIM Student ID: 10256384</small>
  <h1>Poetry Generator</h1>
  <textarea id="poemInput" rows="10" cols="50" placeholder="Enter your poem
here..."></textarea><br>
  <button id="findRhymesButton">Generate Poem</button>
  <div id="outputBox"></div>

  <script src="Program-2.js"></script>
</body>
</html>
```

Youtube video Link:

<https://youtu.be/sLbI9BjSrgw>

Google Drive video link:

<https://drive.google.com/file/d/1D4VzPSYceZcet0kNyOjdZXsZZk8X5VnW/view?usp=sharing>

Program 1 output

By incorporating both JavaScript and HTML 5, I have managed to transform the illustration envisioned in section 2 into reality. As seen below, the image on the left shows the user inserting the input word to find a rhyme for. When the user writes 'cat' into the input box and clicks the search button, the program executes and displays the rhyming words stored in the array into the output box. In this case, outputting 'Rhyming words: fat, rat, splat, bat, hat, chat' as seen in the image on the right.

Name: Bharath Raj Rajamohan
UOL Student ID: 230656932
SIM Student ID: 10256384

Rhyme Finder

Enter a word:

Search

Name: Bharath Raj Rajamohan
UOL Student ID: 230656932
SIM Student ID: 10256384

Rhyme Finder

Enter a word:

Search

Rhyming words: fat, rat, splat,
bat, hat, chat

Likewise in the images below, the image on the left shows what happens when the user writes an unrecognisable word into the input box. The word 'meow' is not a word registered within the array. Hence, it has no associated rhyming words causing the program to write 'No rhymes found for this word.' into the output box as seen in the image on the right.

Name: Bharath Raj Rajamohan
UOL Student ID: 230656932
SIM Student ID: 10256384

Rhyme Finder

Enter a word:

Search

Name: Bharath Raj Rajamohan
UOL Student ID: 230656932
SIM Student ID: 10256384

Rhyme Finder

Enter a word:

Search

No rhymes found for this word.

Program 2 output

Similar to program 1, I have incorporated both JavaScript and HTML 5 for the creation of program 2. As seen below, the image on the left shows the user inserting their poem into the input box with the '[?]' placeholder on the third line to indicate the word in which they need help generating. Once the 'Generate Poem' button is clicked, the program is run and searches all the word in the array loaded from the CSV file for words that end with the same last 2 letters as the reference word which in this case is 'ht' from 'heart'. The image on the right shows the new poem generated by the program. As seen in the third line, the placeholder has been replaced with 'art' as it's last 2 letters are 'ht' just like the reference word 'heart'.

Name: Bharath Raj Rajamohan
UOL Student ID: 230656932
SIM Student ID: 10256384

Name: Bharath Raj Rajamohan
UOL Student ID: 230656932
SIM Student ID: 10256384

Poetry Generator

```
My darling
all i want
is to be your [?]
and show you
all the little stars of my heart
```

Generate Poem



Poetry Generator

```
My darling
all i want
is to be your [?]
and show you
all the little stars of my heart
```

Generate Poem

Processed Poem:

```
My darling
all i want
is to be your art
and show you
all the little stars of my heart
```

The two images below showcases another example but with a different reference word. In this case, 'heart' which was the previous reference word has been changed to 'mine' thereby changing the last 2 letters that will be matched to 'ne'. The picture on the right displays the new poem generated according to the new reference word. The program has now chosen 'alone' to be the new word to replace the '[?]' placeholder in accordance with the updated search parameter being 'ne'. This is an accurate representation of my program's flexibility. The program that I have created is capable of adapting to any poem that the user inputs and finds the most suitable word available to generate the finished poem.

Name: Bharath Raj Rajamohan
UOL Student ID: 230656932
SIM Student ID: 10256384

Poetry Generator

```
My darling  
all i want  
is to be your [?]  
and show you  
all the little stars of mine
```

Generate Poem



Name: Bharath Raj Rajamohan
UOL Student ID: 230656932
SIM Student ID: 10256384

Poetry Generator

```
My darling  
all i want  
is to be your [?]  
and show you  
all the little stars of mine
```

Generate Poem

Processed Poem:

```
My darling  
all i want  
is to be your alone  
and show you  
all the little stars of mine
```

Section 6

A defect found in both programs 1 and 2, is its inability to recognise and process special characters and punctuation. Currently, both programs are capable of converting the input word into lowercase while also trimming them if necessary. However, special characters such as '!' and punctuation such as '.' might cause issues in the execution of the codes and lead to the words not being matched correctly. This can be possibly solved by implementing JavaScript's replace method to replace all non-alphanumeric characters with empty strings.

When taking a detailed look into program 2, more limitations can be identified. Firstly, my CSV file is loaded synchronously along with all other code. My current program is capable of smoothly processing all of them synchronously, however if I decide to upgrade my CSV file to one containing a larger set of data, the algorithm might freeze or become unresponsive while waiting for the data set to finish loading which is not an outcome that should be tolerated. A possible remedy for this would be to enable the program to load the CSV file asynchronously using JavaScript's promise object or async/await function. This would ensure that the program remains responsive while waiting for the CSV file loads in the background, resulting in improved performance and user experience.

Additionally, the current replacement logic is based solely on matching the last 2 letters of the reference words, which may not result in phonetically appropriate replacement words. A remedy for this would be to adopt more sophisticated algorithms such as phonetic matching algorithms (e.g., Soundex) or integrating a rhyming dictionary Application Programming Interface (API). This would consider the phonetic properties of a word while finding suitable rhyming words, allowing the program to generate poems of higher quality.

Lastly, program 2 is made only to operate on poems that follow the 'ABAB' rhyming schemes. Therefore, there are a myriad of other rhyme schemes such as ABBA, ABCB, ABCCBA etc. that program 2 is incapable of aiding on. This makes the program a specialised resource instead of a generalised one.

References

GeeksforGeeks. (n.d.). Array representation in data structures. GeeksforGeeks. Retrieved July 2, 2024, from <https://www.geeksforgeeks.org/array-representation-in-data-structures/>

McMahon, C. (2020, June 10). Linear, binary, and interpolation search algorithms explained. DEV Community. Retrieved July 2, 2024, from <https://dev.to/christinamcmahon/linear-binary-and-interpolation-search-algorithms-explained-55ni>

GeeksforGeeks. (n.d.). How do dynamic arrays work? GeeksforGeeks. Retrieved July 2, 2024, from <https://www.geeksforgeeks.org/how-do-dynamic-arrays-work/>

GeeksforGeeks. (n.d.). Linked list data structure. GeeksforGeeks. Retrieved July 2, 2024, from <https://www.geeksforgeeks.org/data-structures/linked-list/>

TutorialsPoint. (n.d.). Queue data structure. TutorialsPoint. Retrieved July 2, 2024, from https://www.tutorialspoint.com/data_structures_algorithms/dsa_queue.htm

TutorialsPoint. (n.d.). Stack data structure. TutorialsPoint. Retrieved July 2, 2024, from https://www.tutorialspoint.com/data_structures_algorithms/stack_algorithm.htm